SPACEX FALCON9 LAUNCHING ANALYSIS

Scott Yao

March 09, 2026

ScottYao's GitHub URL of the final project:

https://github.com/ScottYaoUnited/Final_Project

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

1:3 Executive Summary

Summary of methodologies

- Data collection
- Data wrangling
- Exploratory Data Analysis (EDA) with data visualization
- Exploratory Data Analysis (EDA) with SQL
- Building an interactive map with Folium
- Building a Dashboard with Plotly Dash
- Predictive analysis (Classification)

Summary of all results

- Exploratory Data Analysis (EDA) results
- Interactive analytics
- Predictive analysis

1:4 Introduction

Basic Background

This is my capstone project for my Data Science courses. In this project, I will practice to use learned data science knowledge to perform a real-world data analysis.

Problem statement

SpaceX advertises Falcon 9 with a cost of 62 million dollars; other providers cost upwards of 165 million dollars each, much of the savings is because SpaceX can reuse the first stages.

Therefore, if we can determine if the first stage will land, we can determine the cost of a launch.

A company call “Space Y” would like to compete with SpaceX. I will provide data analysis to predict if SpaceX will reuse the first stage for Space Y leadeship.

Section 1

Methodology

Methodology

- Data Collection
  - SpaceX REST API endpoints or URL
  - Web Scraping related Wiki pages for Falcon 9 Launch records
- Data wrangling
  - Dealing with null/missing values
  - Remove any unnecessary values from datasets
  - Keep NULL values using one hot encoding

- Perform exploratory data analysis(EDA) using SQL and visualization tool
- Perform interactive visual analytics and Dashboard
- Perform predictive analysis via Classification
  - Build Machine learning pipeline and perform Grid Search
  - Models: Logistic Regression, Support Vector Machines, Decision Tree Classifier, and K-nearest neighbors

Data Collection

We collect data from two resources:

1. Data from SpaceX REST API URL
    - https://api.spacexdata.com/v4/
    - https://api.spacexdata.com/v4/capsules
    - https://api.spacexdata.com/v4/cores
    - https://api.spacexdata.com/v4/launches/past
    - Using GET request and JSON format
    - Convert JSON data to Python pandas data frame object using json_normalize function

2. Web Scraping Falcon 9 Launch Records from Wikipedia pages
    - Using Python BeautifulSoup package to web scrape HMTL tables
    - Parse the data and convert data into a pandas data frame
    - Remove Falcon 1 launches keeping only the Falcon 9 launches
    - Filter data frame using BoosterVersioncolumn

1:5 Data Collection – SpaceX API -1

Task 1: Request and parse the SpaceX launch data using the GET request

To make the requested JSON results more consistent, we will use the following static response object for this project:

```
static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/API_call_spacex_api.json'
```

We should see that the request was successfull with the 200 status response code

```
response=requests.get(static_json_url)
```

```
response.status_code
```

```
200
```

Now we decode the response content as a Json using `.json()` and turn it into a Pandas dataframe using `.json_normalize()`

```
# Use json_normalize meethod to convert the json result into a datafra
response.json()
data = pd.json_normalize(response.json())
```

We will now use the API again to get information about the launches using the IDs given for each launch. Specifically we will be using columns rocket, payloads, launchpad, and cores.

```
# Lets take a subset of our dataframe keeping only the features we want and the flight number, and date_utc.
data = data[['rocket', 'payloads', 'launchpad', 'cores', 'flight_number', 'date_utc']]

# We will remove rows with multiple cores because those are falcon rockets with 2 extra rocket boosters and rows that have multiple payloads in a single rocket.
data = data[data['cores'].map(len)==1]
data = data[data['payloads'].map(len)==1]

# Since payloads and cores are lists of size 1 we will also extract the single value in the list and replace the feature.
data['cores'] = data['cores'].map(lambda x : x[0])
data['payloads'] = data['payloads'].map(lambda x : x[0])

# We also want to convert the date_utc to a datetime datatype and then extracting the date leaving the time
data['date'] = pd.to_datetime(data['date_utc']).dt.date

# Using the date we will restrict the dates of the launches
data = data[data['date'] <= datetime.date(2020, 11, 13)]
#data.head(1)
```

Task 2: Filter the dataframe to only include Falcon 9 launches

Finally we will remove the Falcon 1 launches keeping only the Falcon 9 launches. Filter the data dataframe using the BoosterVersion column to only keep the Falcon 9 launches. Save the filtered data to a new dataframe called data_falcon9.

```
data_falcon9 = data[data['BoosterVersion']!='Falcon 1']
```

Now that we have removed some values we should reset the FlgihtNumber column

```
#data_falcon9.loc[:,'FlightNumber'] = list(range(1, data_falcon9.shape[0]+1))
data_falcon9.loc[:,'FlightNumber'] = list(range(1, data_falcon9.shape[0]+1))
data_falcon9
```

1:5 Data Collection – SpaceX API -2

Task 3: Dealing with Missing Values

Calculate below the mean for the PayloadMass using the .mean(). Then use the mean and the .replace() function to replace np.nan values in the data with the mean you calculated.

```
# Calculate the mean value of PayloadMass column

falcon9_mean = data_falcon9['PayloadMass'].mean()
print(falcon9_mean)
# Replace the np.nan values with its mean value

#data_falcon9['PayloadMass'].replace(np.NaN, falcon9_mean)
# 2. Replace NaN values with the mean
data_falcon9['PayloadMass'] = data_falcon9['PayloadMass'].fillna(falcon9_mean)


data_falcon9.isnull().sum()
```

```
]: FlightNumber       0
   Date               0
   BoosterVersion     0
   PayloadMass        0
   Orbit              0
   LaunchSite         0
   Outcome            0
   Flights            0
   GridFins           0
   Reused             0
   Legs               0
   LandingPad        26
   Block              0
   ReusedCount        0
   Serial             0
   Longitude          0
   Latitude           0
   dtype: int64
```

Now we should have no missing values in our dataset except for in LandingPad. We can now export it to a **CSV** for the next section

```
data_falcon9.to_csv('dataset_part_1.csv', index=False)
```

GitHub reference: 1-jupyter-labs-spacex-data-collection-api. pyinb

1:6 Data Collection – Web Scraping from Wikipedia -1

TASK 1: Request the Falcon9 Launch Wiki page from its URL

#First, let's perform an HTTP GET method to request the Falcon9 Launch HTML page, as an HTTP response.

response = requests.get(static_url,headers=headers)
data = response.text

#Create a BeautifulSoup object from the HTML response

```
response.raise_for_status() # Check for bad response status codes

# Step 2: Create a BeautifulSoup object from the response content
# It is generally better to use response.content (bytes) to avoid
# potential character encoding issues, though response.text (string) also works.
soup = BeautifulSoup(response.content, 'html.parser')

# You can now use the 'soup' object to navigate and extract data
print("BeautifulSoup object created successfully.")
print(f"Page title: {soup.title.string}")
```

BeautifulSoup object created successfully.
Page title: List of Falcon 9 and Falcon Heavy launches - Wikipedia

TASK 2: Extract all column/variable names from the HTML table header

```
html_tables = soup.find_all('table')
#You can then iterate through the list of tables
for table in html_tables:
    print(f"Found a table: {table.name}")
    # Further processing of each table can be done here
```


```
html_tables = soup.find_all('table')
#You can then iterate through the list of tables
for table in html_tables:
    print(f"Found a table: {table.name}")
    # Further processing of each table can be done here
```

1:6 Data Collection – Web Scraping from Wikipedia -2

TASK 3: Create a data frame by parsing the launch HTML tables

We will create an empty dictionary with keys from the extracted column names in the previous task. Later, this dictionary will be converted into a Pandas dataframe

```
launch_dict= dict.fromkeys(column_names)

# Remove an irrelvant column
del launch_dict['Date and time ( )']

# Let's initial the launch_dict with each value to be an empty list
launch_dict['Flight No.'] = []
launch_dict['Launch site'] = []
launch_dict['Payload'] = []
launch_dict['Payload mass'] = []
launch_dict['Orbit'] = []
launch_dict['Customer'] = []
launch_dict['Launch outcome'] = []
# Added some new columns
launch_dict['Version Booster']= []
launch_dict['Booster landing'] = []
launch_dict['Date'] = []
launch_dict['Time'] = []

print(launch_dict)
```

Next, we just need to fill up the launch_dict with launch records extracted from table rows.

```
extracted_row = 0
#Extract each table
for table_number,table in enumerate(soup.find_all('table',"wikitable plainrowheaders collapsible")):
   # get table row
    for rows in table.find_all("tr"):
        #check to see if first table heading is as number corresponding to launch a number
        if rows.th:
            if rows.th.string:
                flight_number=rows.th.string.strip()
                flag=flight_number.isdigit()
```

After you have fill in the parsed launch record values into `launch_dict`, you can create a dataframe from it.

```
#df= pd.DataFrame({ key:pd.Series(value) for key, value in launch_dict.items()})
df= pd.DataFrame(launch_dict)
```

GitHub reference: 2-jupyter-labs-webscraping.ipynb

1:7 Data Wrangling

Need to handle missing values and calculate Task 1~3, add an outcome column in Task 4

TASK 1: Calculate the number of launches on each site

```
# Apply value_counts() on column LaunchSite
df['LaunchSite'].value_counts()
```

```
LaunchSite
CCAFS SLC 40    55
KSC LC 39A      22
VAFB SLC 4E     13
Name: count, dtype: int64
```

TASK 2: Calculate the number and occurrence of each orbit

```
df['Orbit'].value_counts()
```

```
Orbit
GTO      27
ISS      21
VLEO     14
PO        9
LEO       7
SSO       5
MEO       3
ES-L1     1
HEO       1
SO        1
GEO       1
Name: count, dtype: int64
```

TASK 3: Calculate the number and occurrence of mission outcome of the orbits

```
# landing_outcomes = values on Outcome column
landing_outcomes = df['Outcome'].value_counts()
landing_outcomes
```

```
Outcome
True ASDS      41
None None      19
True RTLS      14
False ASDS      6
True Ocean      5
False Ocean     2
None ASDS       2
False RTLS      1
Name: count, dtype: int64
```

TASK 4: Create a landing outcome label from Outcome column

Using the Outcome, create a list where the element is zero if the corresponding row in Outcome is in the set bad_outcome; otherwise, it's one. Then assign it to the variable landing class:

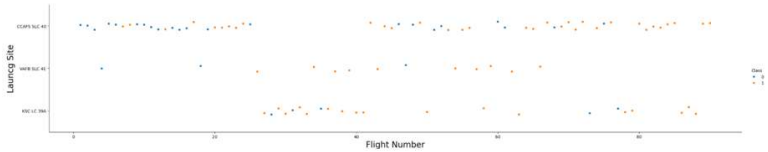
```
df.head()
# landing_class = 0 if bad_outcome
# landing_class = 1 otherwise

landing_class= []
for row in df['Outcome']:
    if row in bad_outcomes:
        landing_class.append(0)
    else:
        landing_class.append(1)
```

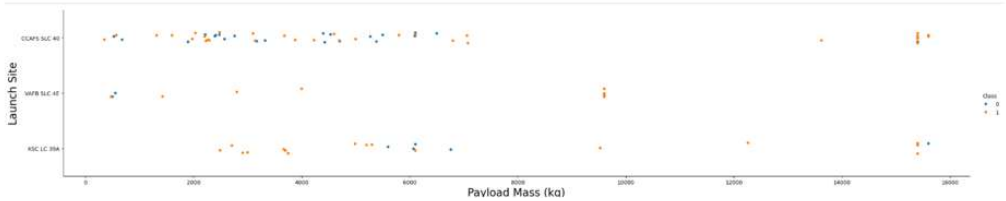
```
df['Class']=landing_class
df[['Class']].head(8)
```

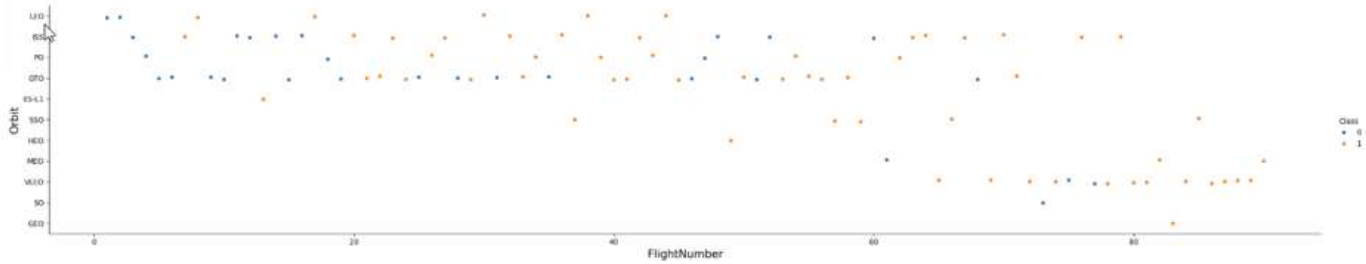
Github reference: 3-labs-jupyter-spacex-Data wrangling.ipynb

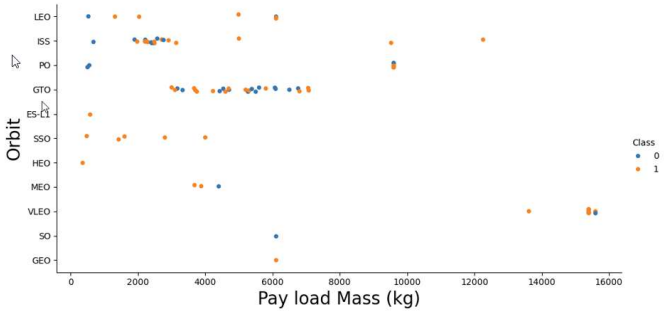
1: 8 Exploratory Data Analysis (EDA) with Data Visualization -1

Scatter point charts visualize relationship between 1) Flight Number and Launch Site 2) Payload and Launch Site 3) Flight Number and Orbit type 4) Payload and Orbit type.





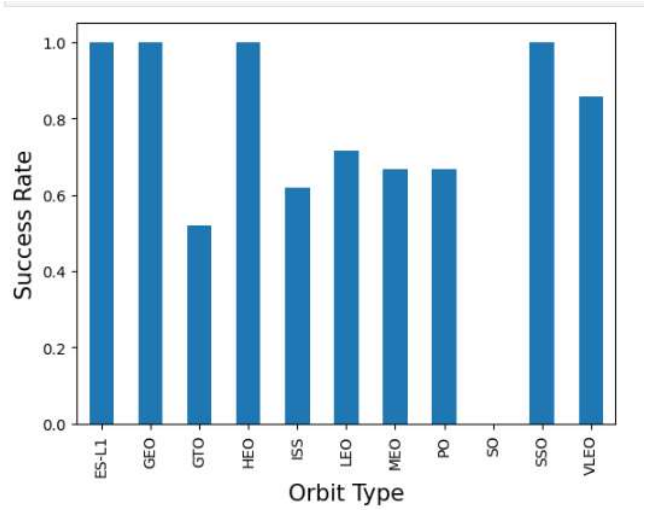


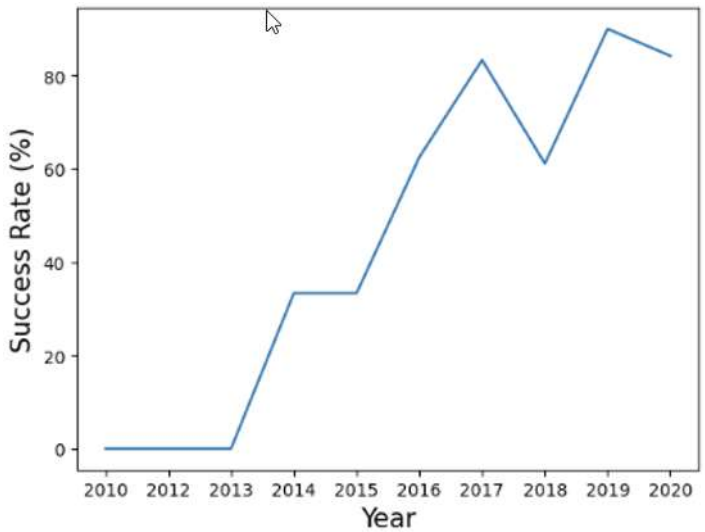


1:8 Exploratory Data Analysis (EDA) with Data Visualization -2

Bar chart visualizes the relationship between success rate and orbit type

Line chart visualizes the average launch success trend





Github reference: 5-EDA-with-data-visualization-lab.ipynb

1:9 Exploratory Data Analysis (EDA) with SQL Summary

I will perform some Exploratory Data Analysis using a database SQL to expose information; then proceeding to explore the relationships between the factors using visualization libraries, such as Matplotlib and Seaborn

Task 1 - Display the names of the unique launch sites in the space mission

Task 2 Display 5 records where launch sites begin with the string 'CCA'

Task 3 Display the total payload mass carried by boosters launched by NASA (CRS)

Task 4 Display average payload mass carried by booster version F9 v1.1

Task 5 List the date when the first successful landing outcome in ground pad was achieved.

Task 6 List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000¶

Task 7 List the total number of successful and failure mission outcomes

Task 8 List all the booster versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

Task 9 List the records which will display the month names, failure landing outcomes in drone ship ,booster versions, launch site for the months in year 2015.

Task 10 Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

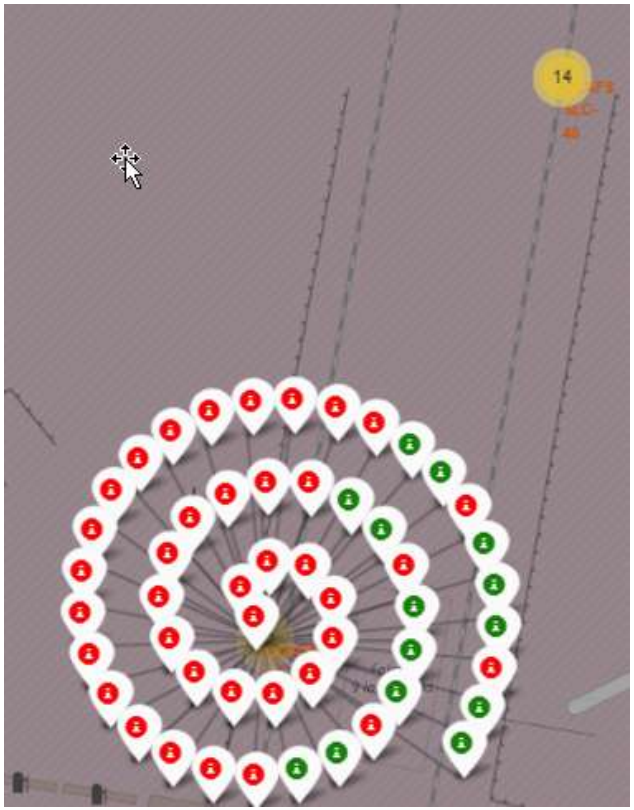
GitHub Reference: 4-jupyter-labs-eda-sql-coursera_sqllite.ipynb

1:10 Interactive Visual Analytics with Folium lab

TASK 1: Mark all launch sites on a map

Task 2: Mark the success/failed launches for each site on the map

TASK 3: Calculate the distances between a launch site to its proximities



GitHub Reference: 6-Interactive Visual Analytics with Folium.ipynb

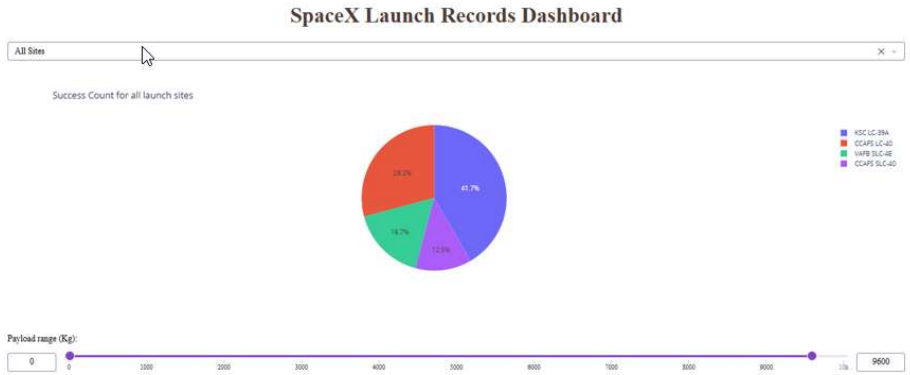
1: 10 Interactive Dashboard with Plotly Dash

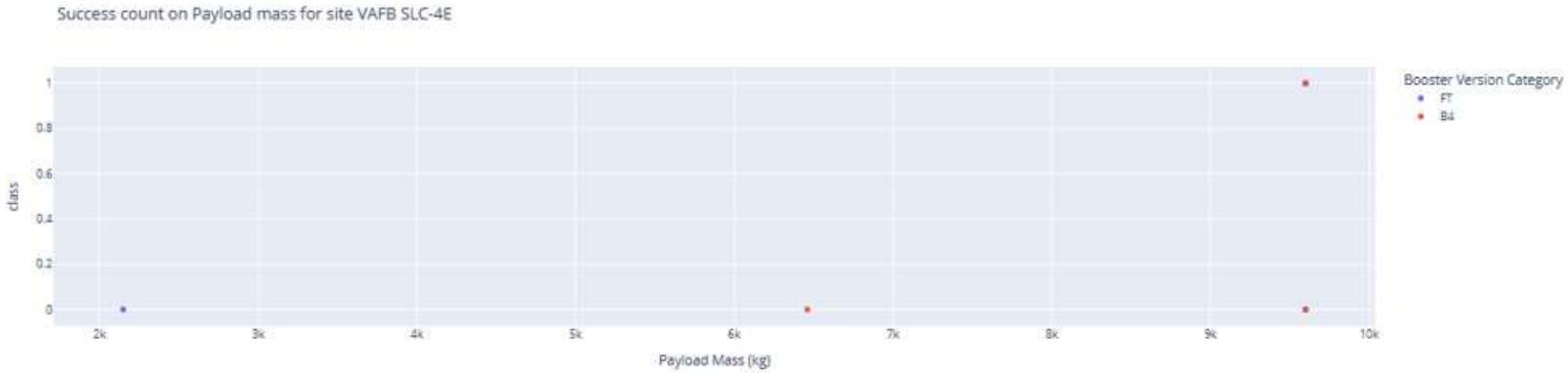
TASK 1: Add a Launch Site Drop-down Input Component

TASK 2: Add a callback function to render success-pie-chart based on selected site dropdown

TASK 3: Add a Range Slider to Select Payload.

TASK 4: Add a callback function to render the success-payload-scatter-chart scatter plot

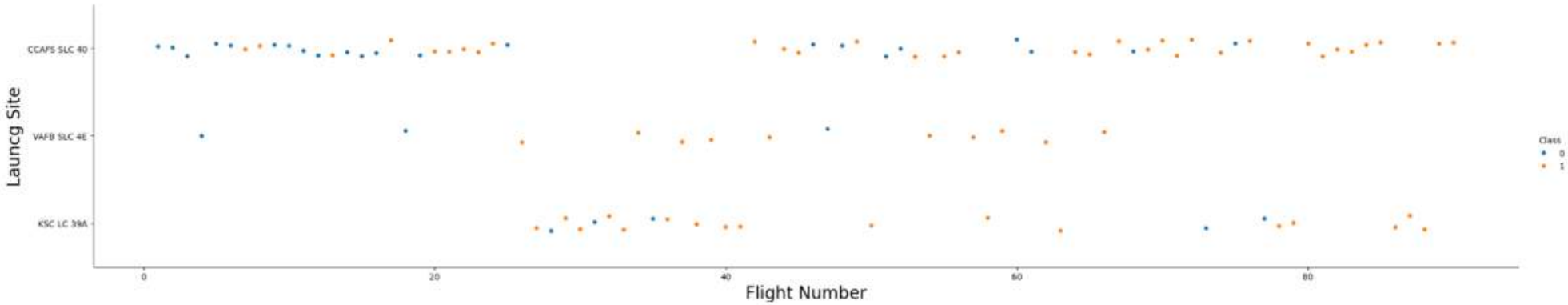




GitHub reference: 7-Lab_Build_an_Interactive_Dashboard_with_Plotly_Dash.py

1:11 Complete all EDA visualization result –scatt plots, bar, charts, yearly trend

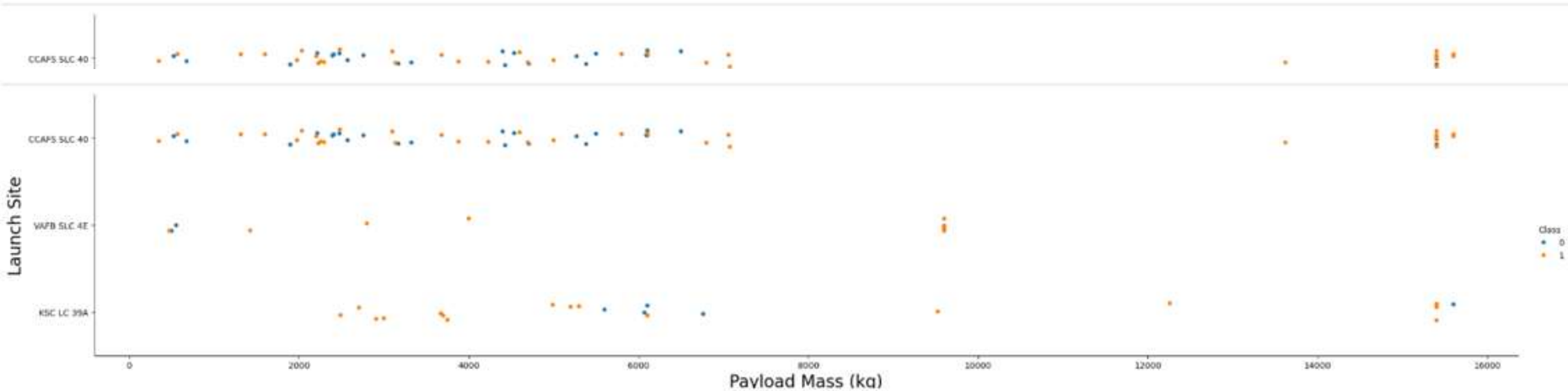
Flight Number vs Launch Site



Use the function catplot to plot Flight Number vs LaunchSite.
Total launch number from site CCAFS CLS 40 are more than WAFBSLC 4E and KSC LC 39A

1:11 Complete all EDA visualization result –scatt plots, bar, charts, yearly trend

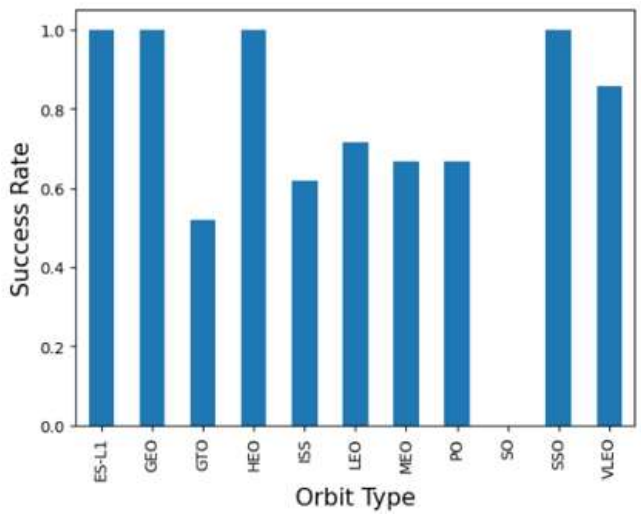
Payload Mass vs Launch Site



- For VAFB-SLC 4E launch site there are no rockets launched for heavy payload mass(greater than 10000).
- Payloads with lower mass have more launches than heavy payloads.

1:11 Complete all EDA visualization result –scatt plots, bar, charts, yearly trend

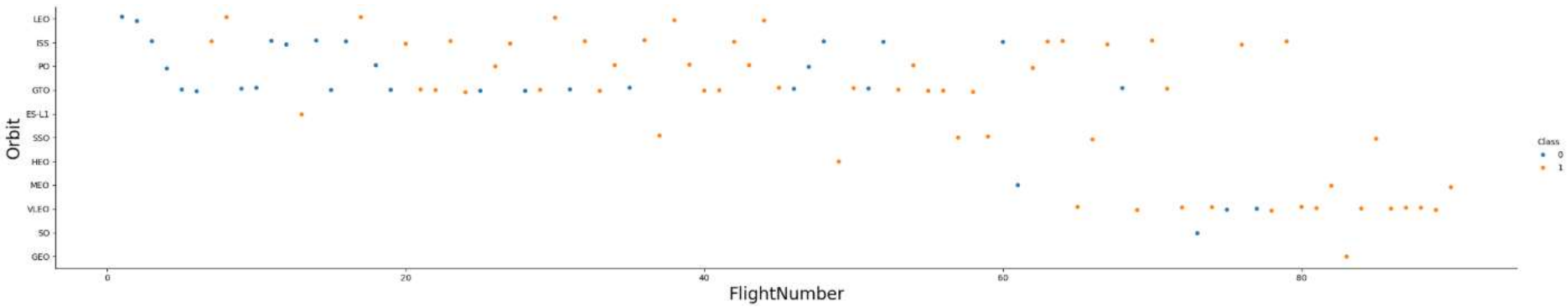
Success rate vs Orbit Type



Orbit types ES-L1,GEO, HEO and SSO have highest success rate

1:11 Complete all EDA visualization result –scatt plots, bar, charts, yearly trend

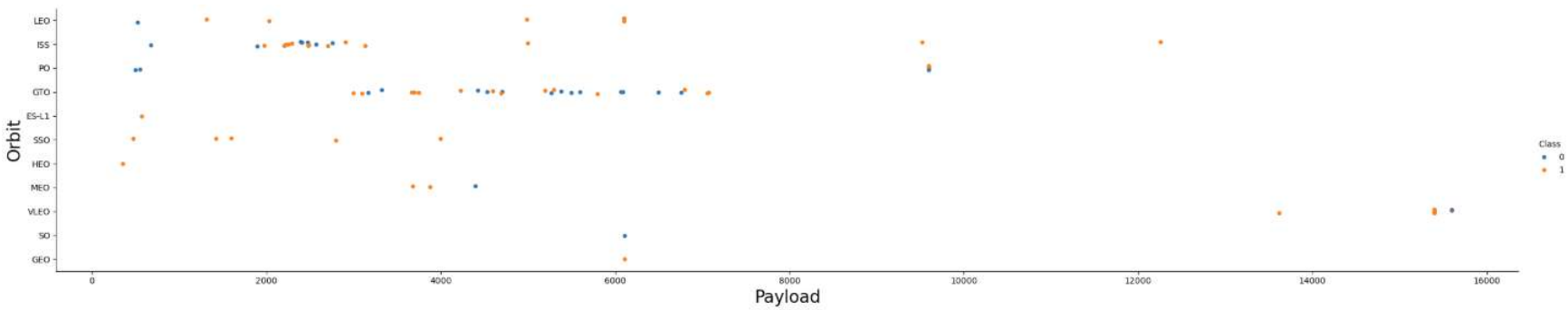
FlightNumber vs Orbit type



LEO, ISS,PO,GTO orbits have more launches between FlightNimber 0 to 60 and VLEO has most launches after FlightNumber 60

1:11 Complete all EDA visualization result –scatt plots, bar, charts, yearly trend

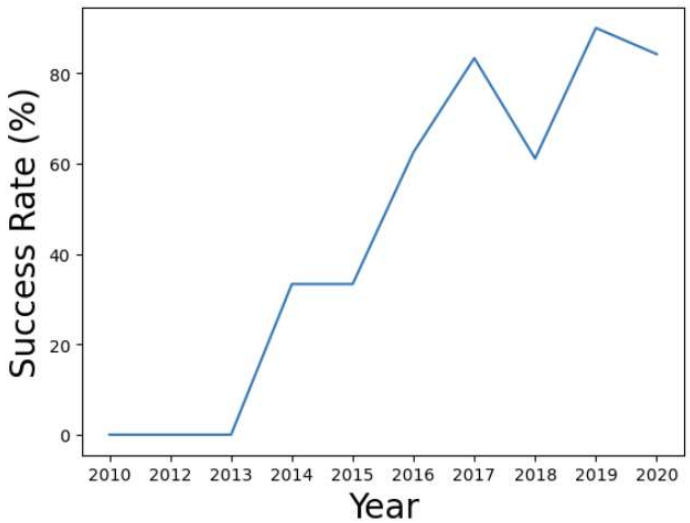
Payload vs Orbit type



With heavy payloads the successful landing or positive landing rate are more for LEO and ISS. However for GTO we cannot distinguish this well as both positive landing rate and negative landing(unsuccessful mission) are both there here.

1:11 Complete all EDA visualization result –scatt plots, bar, charts, yearly trend

Launch success yearly trend



We can observe that the success rate since 2013 kept increasing till 2020

1.12 Completed all required EDA SQL slides – launch sites

Task 1 - Display the names of the unique launch sites in the space mission

```
%sql select distinct Launch_Site FROM SPACEXTBL
```

```
 * sqlite:///my_data1.db
Done.
```

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Task 2 Display 5 records where launch sites begin with the string 'CCA'

```
%sql select * from SPACEXTABLE where Launch_Site like 'CCA%' Limit 5
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

1.12 Completed all required EDA SQL slides – payloads

Task 3 Display the total payload mass carried by boosters launched by NASA (CRS)

```
%sql select sum(PAYLOAD_MASS__KG_) from SPACEXTABLE  where customer = 'NASA (CRS)'
```

```
 * sqlite:///my_data1.db
Done.
```

sum(PAYLOAD_MASS__KG_)
45596

Task 4 Display average payload mass carried by booster version F9 v1.1

```
%sql select avg(PAYLOAD_MASS__KG_) from SPACEXTABLE where Booster_Version = 'F9 v1.1'
```

```
 * sqlite:///my_data1.db
Done.
```

avg(PAYLOAD_MASS__KG_)
2928.4

1.12 Completed all required EDA SQL slides – Success rates

Task 5 List the date when the first successful landing outcome in ground pad was achieved

```
%%sql
SELECT min(Date)
FROM SPACEXTBL
WHERE Landing_Outcome = 'Success (ground pad)';
```

```
 * sqlite:///my_data1.db
Done.
```

min(Date)
2015-12-22

Task 6 List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000¶

```
%sql select distinct Booster_Version from SPACEXTABLE where Landing_Outcome = 'Success (drone ship)' and PAYLOAD_MASS__KG_ > 4000 and PAYLOAD_MASS__KG_ < 6000
```

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

1.12 Completed all required EDA SQL slides – rankings and time analysis

Task 7 List the total number of successful and failure mission outcomes

```
%sql select distinct Mission_Outcome, count(*) from SPACEXTABLE group by Mission_Outcome
```

```
 * sqlite:///my_data1.db
Done.
```

Mission_Outcome	count(*)
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Task 8 List all the booster versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.

```
%sql select Booster_Version from SPACEXTABLE where PAYLOAD_MASS__KG_ = (select max(PAYLOAD_MASS__KG_)from SPACEXTABLE)
```

Booster_Version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

1.12 Completed all required EDA SQL slides – rankings and time analysis

Task 9 List the records which will display the month names, failure landing outcomes in drone ship ,booster versions, launch site for the months in year 2015.

```
%sql select substr(Date, 6,2) as Month, Landing_Outcome, Booster_Version, Launch_Site  from SPACEXTABLE where Landing_Outcome = 'Failure (drone ship)' and substr(Date,0,5) = '2015'
```

Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Task 10 Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

```
%sql select Landing_Outcome, count(*) as 'Count' from SPACEXTABLE where Date between '2010-06-04' and '2017-03-20' group by Landing_Outcome order by Count desc
```

Landing_Outcome	Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

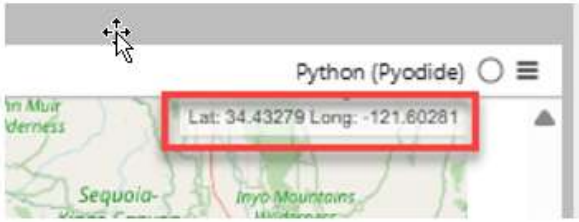
1.13 Complete Folium map with launch markers, launch records and Proximity analysis

Task 1: Mark all launch sites on a map

Task 2: Mark the success/failed launches for each site on the map

1.13 Complete Folium map with Proximity analysis

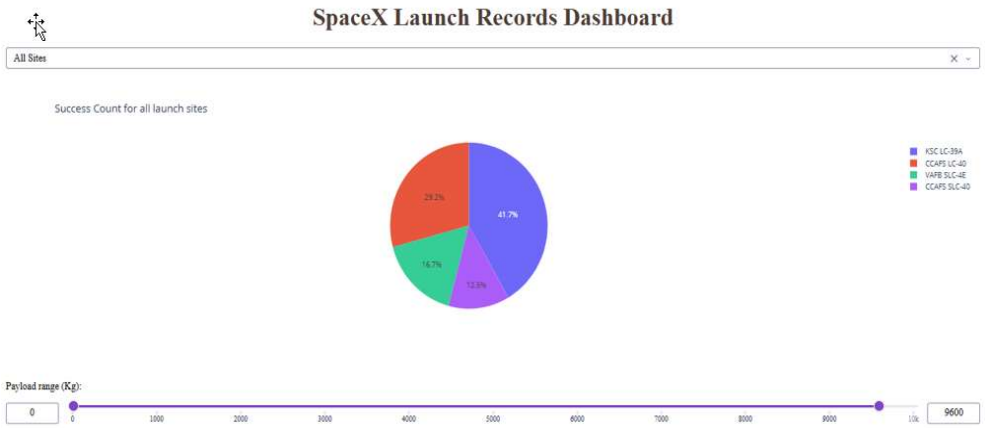
TASK 3: Calculate the distances between a launch site to its proximities



- Get the location coordinates

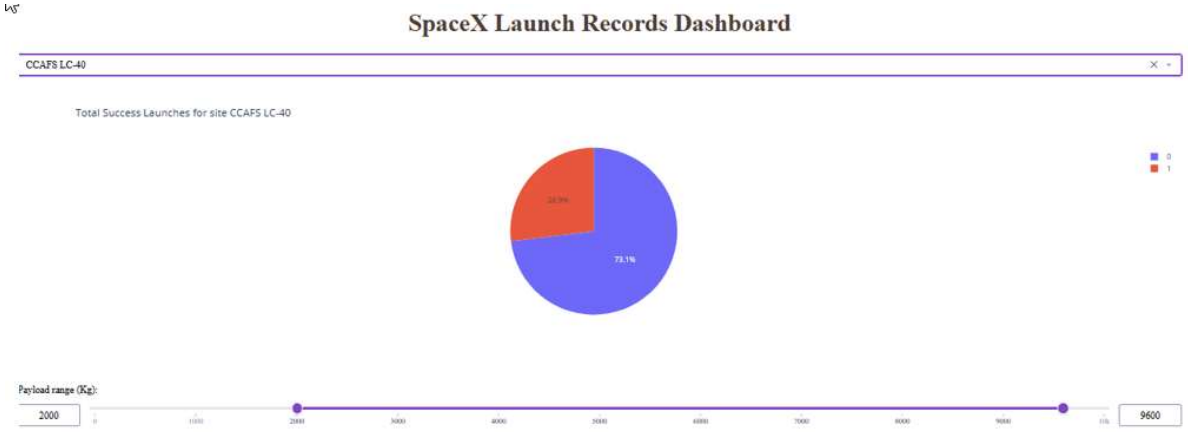
- Calculate the distance from location coordinates

1.14 Complete the Poltly Dash slides with success pie charts

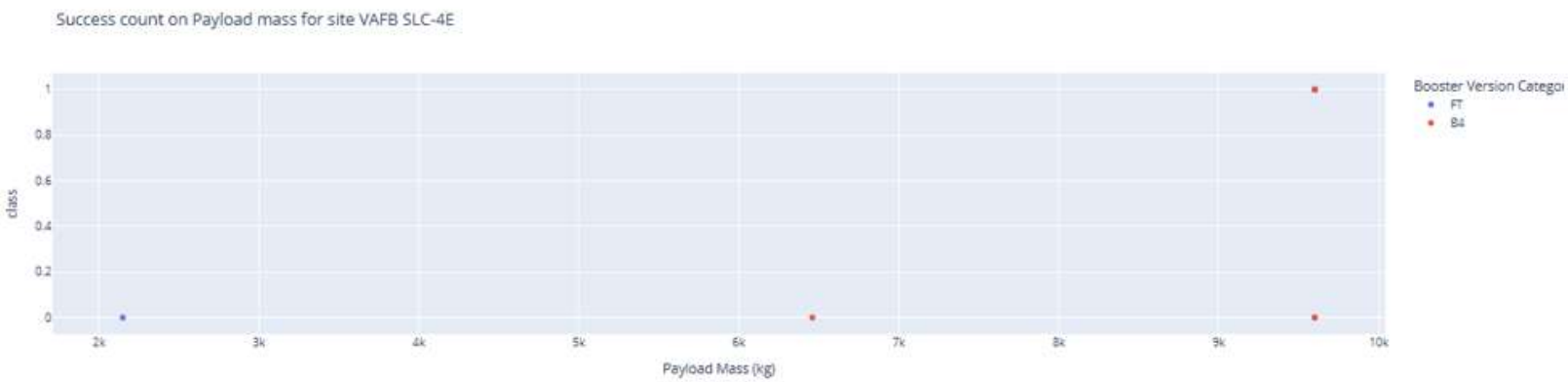


- KSC LC-39 A has a 41.7% success count
- CCAFS LC-40 has a 29.2 %
- VAFB SLC-4E has a16.7%
- CCAFS SLC-40 has a 12.5%

- CCAFS LC-40 has a 26.9% success rate and 73.1 failure rate



1.14 Complete the Poltly Dash slides with payload vs launch outcome scatter plots



VAFB SLC -4E has more success launches on
B4 booster version

1:15 Predictive Analysis (Classification) multiple models, evaluation results, confusion matrix, best model explanation , and a clear conclusion with creative and innovation insights

TASK 12 Find the method performs best:

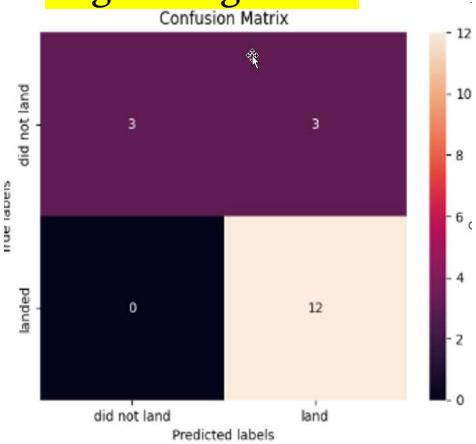
```
Logistic Regression Best Accuracy: 0.8464285714285713
Support Vector Machine Best Accuracy: 0.8482142857142856
Decision Tree Best Accuracy: 0.875
K-Nearest Neighbors Best Accuracy: 0.8482142857142858
The best performing model is Decision Tree with an accuracy of 0.8750
```

- The best performing model is Decision Tree with 87.5% accuracy

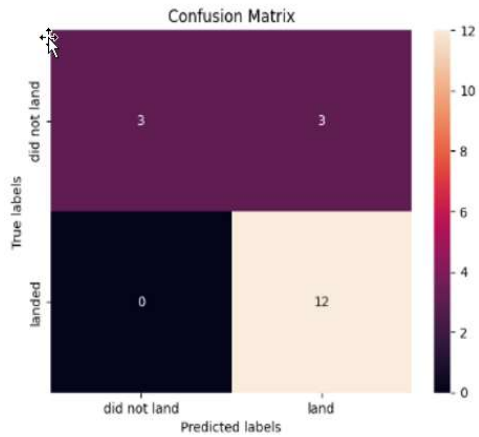
GitHub Rederence:8-SpaceX_Machine Learning Prediction_Part_5.ipynb

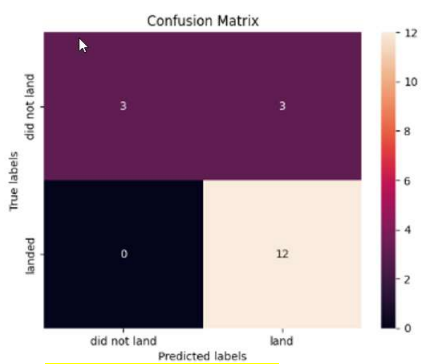
1.15 Confusion Matrix

Logistic regression

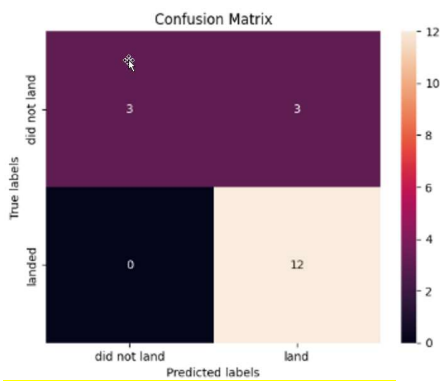


support vector machine





Decision Tree



support vector machine

LR, SVM, DT and KNN model have same successful landings and same confusion matrix

1.15 Conclusion

- Decision Tree is the best performing model
- CCAFS SLC 40 has more launch times
- Orbit types ES-L1,GEO, HEO and SSO have highest success rate
- SpaceX launch success rate increases with more year launching experience
- KSC LC-39A has the most successful launch counts

Questions?

Thank You!